RTX 4090 DiT fp8 / bf16 / fp16 추론 실험

deployment_plan.md 2장 "4090에서 DiT fp8/fp16 실험"의 결과다.

요약

- **fp8 연산(torchao) + `torch.compile`이 가장 빠르고 VRAM도 가장 적다**: 1.46초/장, 피크 9.6GB (bf16 2.20초/13.5GB 대비 34% 빠름, 3.9GB 절약).
- fp8 연산은 **compile 없이는 이득이 거의 없다** (2.17초, bf16과 비슷).
- 기존 `--fp8-weights`(fp8 저장 + bf16 연산)는 VRAM만 3.2GB 줄고 속도는 3.6% 느리다. 4090의 fp8 텐서코어를 쓰지 않는다.
- 품질: 모든 설정이 bf16과 육안상 동일하다. 단 sample17(극단적 결손)은 수치 차이에 민감해 설정마다 결과가 갈린다.
- 24GB 안에 여유 있게 들어간다. `--cpu-offload`는 필요 없다.

1. 실험 조건

항목	값
GPU	RTX 4090 24GB (Ada, compute capability 8.9)
모델	FLUX.2 klein-4B (distilled), 4 step, seed 0, guidance 1.0
경로	`design_tune_server.py` 실제 HTTP API (`/api/v1/infer`)
스타일	`smile_final`, layout `patient_only`, preset `whitening_only` / `align_whiten`
마스크	`mode: inpaint`, `regions: [mouth]`, dilate 16, feather 12 (final_report.md 예시와 동일)
임베딩	`smile_final.yaml`을 서버 build API로 빌드한 캐시 임베딩 (patient_only, 14초). 추론 시 텍스트 인코더는 로드하지 않는다
환자	sample17, teeth01, yellowteeth, sample18, teeth04 (final_report와 동일), 설정당 10장
환경	torch 2.9.0+cu128, torchao 0.14.1

입력 해상도

서버는 원본 비율을 유지하고, 면적이 `--max-generation-area`(기본 1024×1024)를 넘을 때만 축소한다(업스케일 없음). 16의 배수로 맞춘다.

환자	원본	생성 해상도
sample17	1087×820	1088×816
teeth01	303×346	304×352
yellowteeth	1024×1024	1024×1024

환자	원본	생성 해상도
sample18	709×1030	704×1024
teeth04	1024×1024	1024×1024

피크 VRAM은 1024×1024(yellowteeth, teeth04) 요청에서 나온 값이다 — 서버가 허용하는 최대 해상도다.

2. 비교한 설정

이름	서버 옵션	설명
bf16	`--dtype bf16 --no-fp8-weights`	기준
fp16	`--dtype fp16 --no-fp8-weights`	
fp8 저장	`--dtype bf16 --fp8-weights` (기존 기본값)	가중치 fp8 보관, 레이어마다 bf16으로 올려 계산 (`enable_layerwise_casting`)
fp8 연산	`--dtype bf16 --fp8-compute`	torchao `Float8DynamicActivationFloat8WeightConfig(PerRow)`: 가중치·활성화 모두 fp8, `_scaled_mm`
bf16 + compile	`--dtype bf16 --no-fp8-weights --compile`	
fp8 연산 + compile	`--dtype bf16 --fp8-compute --compile`	

`--fp8-compute`, `--compile`은 이번에 서버에 추가한 옵션이다. fp8 연산에서도 attention(SDPA)은 bf16이고, Linear만 fp8로 계산한다.

3. 결과

속도 / VRAM

설정	평균(10장)	1024×1024	704×1024	304×352	로드 후 VRAM	피크 VRAM
bf16	2.20초	3.00초	2.04초	0.37초	8.4GB	13.5GB
fp16	2.25초	3.10초	2.06초	0.39초	8.4GB	13.5GB

설정	평균(10장)	1024×1024	704×1024	304×352	로드 후 VRAM	피크 VRAM
fp8 저장	2.28초	3.09초	2.12초	0.45초	5.2GB	10.3GB
fp8 연산	2.17초	2.96초	2.01초	0.40초	5.0GB	11.1GB
bf16 + compile	1.91초	2.59초	1.77초	0.34초	—	13.2GB
fp8 연산 + compile	**1.46초**	**1.99초**	**1.35초**	**0.25초**	—	**9.6GB**

- 시간은 클라이언트에서 잰 요청 왕복 시간(업로드, 마스크 생성, VAE, 4 step DiT, 응답 포함)이다.
- VRAM은 `nvidia-smi`의 GPU 전체 사용량이다. DiT + VAE + CUDA context(약 0.3~0.5GB) + PyTorch allocator 캐시가 포함된다. 텍스트 인코더(Qwen3)는 추론 서버에 로드되지 않는다.
- compile 설정은 5개 해상도로 warmup을 끝낸 뒤 측정했다. allocator 캐시가 이미 잡혀 있어 로드 후와 피크 구분이 의미 없다.

transformer 단독 (1024×1024 + 참조 1장 + 텍스트 512 토큰, 1 forward)

설정	ms/forward
bf16 eager	602
fp8 연산 eager	591
bf16 compile	490
fp8 연산 compile	**346** (bf16 eager 대비 1.74배)

eager에서는 활성화 양자화·스케일 계산이 별도 커널로 돌아 fp8 텐서코어 이득을 상쇄한다. compile이 이를 matmul과 fuse해야 빨라진다. compile mode `default`와 `max-autotune-no-cudagraphs`는 속도가 같았다 (348 vs 346ms).

품질



원본 | fp8 저장 | bf16 | fp16 (`align_whiten`).

original bf16 fp8 compute (compile)

teeth01 fp8 vs bf16 diff=0.14



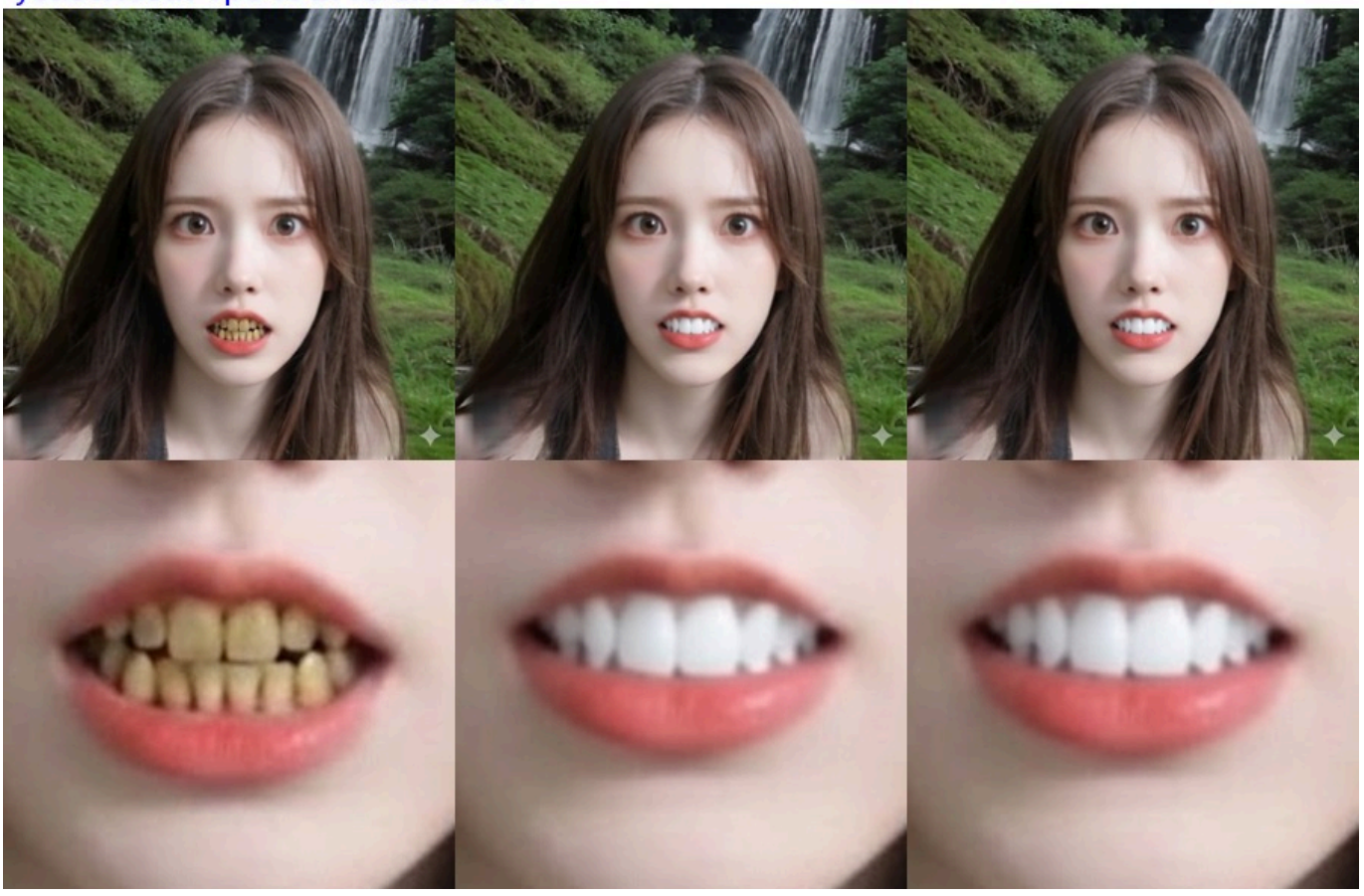
sample18 fp8 vs bf16 diff=0.06





teeth04 fp8 vs bf16 diff=0.02

yellowteeth fp8 vs bf16 diff=0.04





원본 | bf16 | fp8 연산 + compile (`align_whiten`, 아래 줄은 입 확대).

bf16 대비 이미지 전체 평균 픽셀 차이(0-255):

설정	whitening_only	align_whiten
fp16	0.01 ~ 0.08	0.01 ~ 0.47
fp8 저장	0.02 ~ 0.19	0.02 ~ 1.04
fp8 연산 + compile	—	0.02 ~ 0.14 (sample17 제외 4명)

- 4명은 설정 간 차이가 육안으로 구분되지 않는다. fp16도 NaN/검은 이미지 없이 정상이다.
- **sample17 `align_whiten`은 설정마다 다르게 나왔다**: fp8 저장은 틈을 다 메웠고, bf16은 일부, fp16은 틈이 남았다. 어느 정밀도가 낫다기보다 이 케이스가 작은 수치 차이에 민감하다는 뜻으로 본다. 결론을 내리려면 seed 여러 개로 다시 봐야 한다.

4. 운영 시 주의

- `--fp8-compute`는 `torchao`가 필요하고(`pip install torchao==0.14.1`, torch 2.9 기준), compute capability 8.9 이상 (Ada: 4090/L4/L40S, Hopper: H100)에서만 동작한다. A6000/A10G/A100(Ampere)은 기존 `--fp8-weights`를 쓴다.
- `--compile`은 서버 시작 후 **첫 요청에서 한 번** 컴파일한다(fp8 연산 기준 약 250초, bf16 약 70초). `dynamic=True`라 해상도가 바뀌어도 다시 컴파일하지 않는다 — 16가지 해상도로 확인했다. 다만 transformer 호출 인자가 다른 MICE(`mask.mode: mice`)는 첫 요청에서 따로 컴파일한다. 자세한 측정과 대응(regional compile, warmup)은 robustness_report.md 3장. (처음 작성 시 "해상도마다 컴파일한다"고 적었는데 틀렸다 — 5개 해상도 warmup 시간의 대부분이 첫 요청 하나였다.)
- compile 중 `triton_mm ... out of resource` 에러 로그가 나오는데, autotune 후보 중 4090 shared memory 한계를 넘는 것이 버려지는 것이고 결과에는 영향이 없다.
- 추가 여지: attention까지 fp8로 바꾸면(예: fp8 SDPA/FlashAttention-3 계열) 더 빨라질 수 있다 — 이번에는 Linear만 fp8.

5. 부수 발견: `mouth` 마스크는 입술까지 다시 그린다

yellowteeth 결과에서 입술 색이 바뀌는 문제가 보였다. 정밀도와는 무관하다(bf16/fp16/fp8 모두 동일하게 발생).

- `mouth` = 윗입술 + 아랫입술 + 입안 (`my/smile_design/face_mask.py`). dilate 16이면 입 주변 피부까지 덮는다.
- 그 영역 전체를 inpaint로 다시 생성하면서 치아 프롬프트("white")가 입술에도 적용된다. yellowteeth `whitening_only`에서는 아랫입술이 연두/흰색으로 물들었다.



입술 영역 평균 픽셀 변화(0-255, fp8 연산 서버):

영역	yellowteeth whitening	yellowteeth align_whiten	teeth01 whitening	teeth01 align_whiten
`mouth` d16 f12	32.3	26.1	7.7	6.6
`inner_mouth` d4 f6	3.3	2.5	4.3	4.2
`teeth` d4 f6	1.4	1.1	3.2	3.0

- `inner_mouth`(입술 안쪽 윤곽선으로 둘러싼 영역)는 입술을 보존하면서 치아를 전부 바꾼다.
- `teeth`(inner_mouth 안에서 밝고 붉지 않은 픽셀만 고른 색 기반 마스크)는 입술 변화가 가장 적지만, 어두운 치아를 놓친다(yellowteeth 왼쪽 끝).

- 치아 편집은 `inner_mouth`를 쓴다. **서버 기본값을 `--mask-regions inner_mouth --mask-dilate 4 --mask-feather 6`으로 바꿨다** (이전 `mouth` / 8 / 12). `design_tune_client.py mask`의 기본 영역과 `smile_final.yaml`·`smile_teeth.yaml`의 요청 예시도 같이 바꿨다.
- 새 기본값으로 5명을 다시 돌려 5명 모두 입술이 보존되는 것을 확인했다. 단 **sample17의 치아 틈은 `inner_mouth`에서는 메워지지 않았다** (`mouth` d16에서는 메워졌다). 영역이 좁아 치아 배열을 크게 바꿀 여지가 줄어든 것으로 보인다 — 극단적 결손 케이스는 별도 확인이 필요하다.